

Linux Privilege Escalation — POC Report

Intern Name: Ruby Kannaujiya

Intern ID: 309

Tool Name / Topic

Linux Privilege Escalation (techniques & tools: sudo, SUID binaries, kernel exploits, misconfigurations)

History

Privilege escalation techniques have been part of Unix/Linux security research since the early days of multi-user systems. As Linux became ubiquitous (servers, cloud, IoT), attackers and defenders have catalogued common misconfigurations and vulnerable binaries that allow escalation from a low-privileged shell to root. This week focuses on reproducible, safe PoCs for learning.

Description

Privilege escalation on Linux is the process of gaining higher privileges (usually root) from a lower-privilege account by exploiting misconfigurations, weak file permissions, SUID binaries, cron jobs, kernel vulnerabilities, or poorly configured services. The goal is to demonstrate defensible, observable steps and document remediation.

What Is This Topic About?

- Identifying Windows-of-opportunity on a Linux host that allow privilege gain.
- Performing controlled PoCs (non-destructive) to show how access can be escalated.
- Documenting commands, outputs, and mitigations.

Key Characteristics / Features

- Focus on local escalation (not remote exploit).
- Uses standard Linux commands and small proof binaries/scripts.
- Emphasizes enumeration first (information gathering).
- Includes common vectors: SUID, sudo misconfigurations, writable files, cron jobs, kernel exploits, PATH hijacking.
- Reproducible in isolated lab VMs (e.g., Ubuntu, Kali, Metasploitable).

Types / Modules / Vectors Covered

- **Enumeration Module** — `whoami, id, uname, ps, ss, sshd, env, sudo -l`
- **SUID Binary Module** — `find / -perm -4000 -type f 2>/dev/null`
- **Cron & Scheduled Tasks Module** — `ls -la /etc/cron*, user cron (crontab -l)`
- **World-writable Files/Dirs Module** — `find / -writable -type d 2>/dev/null`
- **Service Misconfig Module** — writable service config files, systemd unit overrides
- **Kernel Exploit Module** — kernel version discovery + safe note (do not run untrusted public exploits on production)
- **PATH/Environment Module** — PATH hijacking via scripts run by root

How Will This Help?

- Train you to systematically find escalation paths.
- Teach defenders how attackers pivot; helps to harden systems.
- Provide reproducible reports for audits and labs.
- Prepare documentation and remediation steps for system owners.

Detailed POC (Step-by-step, fully reproducible in a lab)

Lab setup note: Use an isolated VM (e.g., Ubuntu 20.04) snapshot. Do not run exploit code on production systems.

A. Enumeration (always start here)

Commands and expected outputs (capture screenshots):

```
# Current user and groups
whoami
id
```

```
# Kernel & OS info
uname -a
cat /etc/os-release
```

```
# List running services and processes
ps aux --no-heading | head -n 25
```

```
# Open ports
ss -tulpn
```

```
# Check sudo rights
sudo -l
```

What to look for: users with sudo rights, kernel version, processes running as root, services with writable configs, presence of docker, snap, mounts, or unusual binaries.

B. SUID Files (common vector)

Find SUID binaries:

```
find / -perm -4000 -type f 2>/dev/null
```

PoC Example: vulnerable SUID binary `./vuln_suid` (lab created)

1. Create a tiny program (in lab) to simulate SUID (only for lab):

```
// vuln.c – compile in lab
#include <stdlib.h>
int main(){ system("/bin/sh"); return 0; }
```

```
gcc vuln.c -o vuln_suid
chmod 4755 vuln_suid    # sets SUID bit
./vuln_suid             # spawns root shell if owned by root
```

2. **Capture:** show `ls -l vuln_suid` and the shell prompt before/after.

Mitigation: Do not set SUID unless necessary; audit SUID binaries; restrict ownership and remove SUID when not required.

C. Misconfigured sudo (very common)

Check `sudo -l` for allowed commands:

```
sudo -l
```

PoC Example: If sudo allows running `/usr/bin/vim` as root:

```
sudo vim -c '!/bin/sh' # opens shell as root (poC)
```

Or using `sudo -s` if allowed.

Mitigation: Use least privilege in sudoers, avoid allowing editors or shells without restrictions; use NOEXEC or restrict to specific scripts only.

D. Writable Files / World-writable Binaries

Search for writable files owned by root but writable by others:

```
find / -writable -type f 2>/dev/null | head -n 30  
find / -perm -2 -type f 2>/dev/null # world-writable files
```

PoC: If `/usr/local/bin/somescript` owned by root is writable, modify to spawn root shell (lab only). Demonstrate editing the file and getting root when that script is executed by a root process or startup.

Mitigation: Harden file permissions, remove world-writable flags, audit file ownership.

E. Cron Jobs & Scheduled Tasks

Inspect cron jobs:

```
# System cron  
ls -la /etc/cron*  
cat /etc/crontab
```

```
# Per user cron  
crontab -l # for current user; check /var/spool/cron/crontabs
```

PoC Example: If a cron job runs a script from a world-writable directory, replace that script to escalate in lab. E.g., cron runs `/usr/local/bin/backup.sh` as root — if `/usr/local/bin` is writable, attacker can insert payload.

Mitigation: Ensure scripts executed by cron are in secure locations with tight permissions; avoid running user-editable scripts as root.

F. PATH Hijacking

If a root process runs a script that calls `python` or some binary assuming `PATH`, and a writable directory earlier in `PATH` exists:

```
echo $PATH
# Create malicious 'ls' in /tmp and set PATH ahead of /bin in cron-run
context
```

PoC: Demonstrate by creating fake binary in writable directory and show it executing in root context (lab only).

Mitigation: Use absolute paths in scripts run by root; sanitize environment variables.

G. Kernel Exploit (theoretical / safety note)

1. Discover kernel version:

```
uname -r
```

2. **Do not** run untrusted public exploits on production. In lab, known safe exploit PoCs (from authoritative sources) can be used to demonstrate concept — but this is risky.

Mitigation: Keep kernel updated, apply patches, use security modules (SELinux/AppArmor).

H. Example Full PoC Flow (safe, non-destructive)

1. `whoami` → `ruby`

2. `sudo -l` → allowed to run `/usr/bin/nmap` as root
3. Use `nmap` allowed => `sudo nmap --script=/path/to/script` → if script execution allows command execution, craft script to spawn a root shell (lab only).
4. Document exact commands, outputs, and screenshots.

PoC Images (placeholders — add your lab screenshots)

- Terminal: output of `find / -perm -4000 -type f`
- `sudo -l` output screenshot
- Example `ls -l` showing SUID bit (`-rwsr-xr-x`)
- Cron file screenshot showing vulnerable entry
- Before/after screenshot of root shell prompt

15-Liner Summary

- Start with enumeration.
- Check `sudo -l`, SUID, writable files, cron.
- SUID binaries can give root if misused.
- `sudo` misconfigs allow command abuse.
- Writable scripts executed by root are dangerous.
- PATH hijacking can be exploited.
- Kernel exploits are high-impact but risky.
- Use lab VMs for PoCs.
- Document every command & output.
- Capture screenshots for evidence.
- Recommend removing unnecessary SUIDs.
- Harden cron and script permissions.
- Patch the kernel and services.
- Use AppArmor/SELinux for mitigation.
- Regularly audit system for anomalies.

Time to Use / Best Case Scenarios

- During system hardening reviews.
- Post-compromise root cause analysis.
- Red team training and blue team detection exercises.
- Vulnerability assessments in labs.

When to Use During Investigation

- After detecting suspicious local shell.
- When unusual root activity is suspected.
- When user reports unexpected privilege escalation.
- During capture-the-flag / learning labs.

Best Person to Use & Required Skills

User: Pentester / System Administrator / Forensics Investigator

Skills Required:

- Comfortable with Linux CLI (bash).
- Understanding of file permissions and ownership.
- Basic C for compiling small test binaries (lab).
- Knowledge of cron, systemd, and sudoers.
- Familiarity with kernel versioning and patching.

Flaws / Suggestions to Improve (Topic Limitations)

- Some PoCs require creating SUID or modifying files — only safe in lab.

- Kernel exploit testing is dangerous on production.
- Detection depends on logging quality; poor logs hinder investigation.
- Some misconfigurations are environment specific; not all PoCs generalize.

Good About This Topic (Why Learn It)

- Teaches defenders what to look for when hardening hosts.
- Practical and reproducible in an isolated environment.
- Helps build incident response playbooks.
- Immediate mitigation steps often available (fix perms, update sudoers).
- High learning value for system security.

Remediation Checklist (Short)

- Remove unnecessary SUID bits: `find / -perm -4000 -type f` → review.
- Harden sudoers (use `visudo`, avoid `ALL=(ALL) NOPASSWD: /bin/bash`).
- Use absolute paths in scripts and cron.
- Remove world-writable files and directories.
- Patch kernel & OS regularly.
- Enable AppArmor/SELinux and enforce policies.
- Monitor logs and configure alerting for suspicious sudo/SUID usage.